

Práctica 2: PSD DE SEÑALES ALEATORIAS (GNURADIO)

Nestor Santiago Ulloa Reyes – 2215739

Andrés Felipe Quintero Lozano - 2204655

https://github.com/nsantiagou/Comm2_LabA1_G2_Ulloa_Quintero/tree/practica-2

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones

Universidad Industrial de Santander

Abstract

This laboratory practice focused on the study of the Power Spectral Density (PSD) of different signals using GNU Radio. First, bipolar random binary signals were generated with varying *Samples per Symbol* (SPS) values, and their behavior was analyzed in both time and frequency domains. The effect of SPS on the time representation and on the concentration of spectral components was highlighted. Then, white noise was examined, confirming its random appearance in the time domain and its flat PSD across the frequency domain. Finally, real-world data sources were introduced through JPG (image) and WAV (audio) files. Results showed that the PSD of the image data tends to concentrate energy at DC due to long binary runs, while the audio file presented a more balanced spectral distribution with visible harmonic components. The practice was complemented with control questions, allowing the theoretical concepts of sampling, filtering, and modulation to be reinforced through experimental evidence.

1 Introducción

El análisis espectral de señales constituye una herramienta esencial en comunicaciones, ya que permite comprender cómo se distribuye la energía de una señal en el dominio de la frecuencia. La densidad espectral de potencia (PSD) es particularmente relevante en el estudio de señales aleatorias, al facilitar la relación entre sus características temporales y espectrales. En esta práctica de laboratorio, desarrollada en GNU Radio, se abordaron tres ejes principales: primero, la simulación de señales binarias aleatorias bipolares variando el parámetro *Samples per Symbol* (SPS), con lo cual se observó la influencia del tren de impulsos sobre la forma temporal y la nitidez de la PSD. Segundo, la comprobación del ruido blanco en tiempo y frecuencia, verificando su comportamiento aleatorio y su espectro plano característico. Finalmente, se emplearon fuentes reales de información mediante archivos JPG y WAV, evidenciando diferencias en la PSD: en la imagen, la energía se concentró en la componente continua, mientras que en el audio se mantuvo una

distribución más variada y rica en componentes armónicos. El trabajo se complementa con preguntas de control, orientadas a reforzar conceptos de muestreo, filtrado, codificación y modulación, integrando de manera práctica la teoría vista en clase.

2 Metodología

Señal binaria aleatoria bipolar

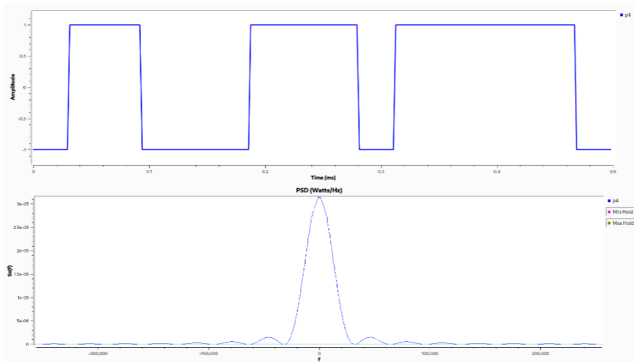
En este laboratorio trabajamos con un SDR y la idea era experimentar de manera práctica cómo se generan y procesan señales digitales. Lo primero que hicimos fue armar el sistema con los bloques y mirar bien cómo se conectaban entre ellos. Nos llamó mucho la atención el filtro de interpolación, porque es el que se encarga de aumentar la tasa de muestreo y sin eso no se podría observar ni analizar la señal como se debe.

Cuando ya entendimos cómo estaba diseñado el sistema, pasamos a generar la señal binaria aleatoria bipolar. En pocas palabras, construimos una secuencia de unos y ceros, pero en vez de dejarlos así, los representamos con voltajes opuestos: +1 y -1. Luego empezamos a mover parámetros como la tasa de bits, la frecuencia de muestreo y el ancho de banda. Lo más interesante fue ver cómo la relación entre las muestras por símbolo cambiaba tanto la forma de la señal en el tiempo como su espectro en frecuencia.

Al final decidimos meterle ruido blanco para ver cómo se afectaba la señal. Como este ruido está distribuido de manera uniforme en todas las frecuencias, lo que buscábamos era observar cómo se comportaba tanto en el tiempo como en la frecuencia. Fue curioso ver que, aunque el ruido parece completamente aleatorio, en el espectro se ve plano, como equilibrado. Eso nos ayudó a entender mejor cómo interactúa con la señal que habíamos generado y por qué siempre hay que tener en cuenta su influencia en sistemas reales.

Configuración de bloques en GNURadio para el análisis de ruido blanco

la siguiente figura nos representa la salida del flujograma para generar y analizar el ruido blanco en el tiempo y su respectiva PSD.

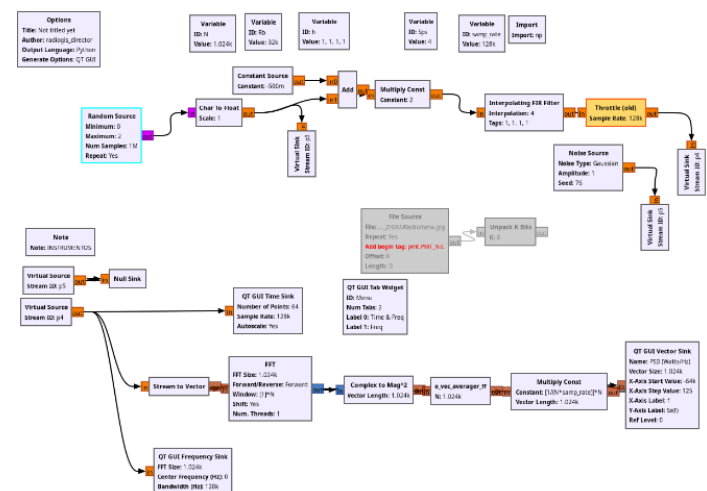


Señal bipolar y su PSD.

Luego seguimos con la integración de señales reales. Para eso, lo que hicimos fue cambiar el bloque Random Source por el bloque File Source, tal como se muestra en la figura. De esa manera, en vez de generar datos aleatorios, empezamos a trabajar con información que venía de archivos, lo que nos permitió ver cómo se comportaba el sistema con señales que ya no eran simuladas, sino tomadas de un entorno más cercano a lo real.

¿Qué bloques se utilizaron para cargar archivos reales?

En esta parte incluimos la figura donde aparecen los bloques que usamos para cargar y procesar tanto la imagen de la rana como el archivo de audio. Con esto pudimos trabajar directamente con archivos reales: primero cargamos una imagen en formato JPG de una rana y, después, un archivo de audio en formato WAV. La idea era ver cómo el sistema respondía con diferentes tipos de datos, no solo señales binarias, y así experimentar con información más cercana a aplicaciones prácticas.

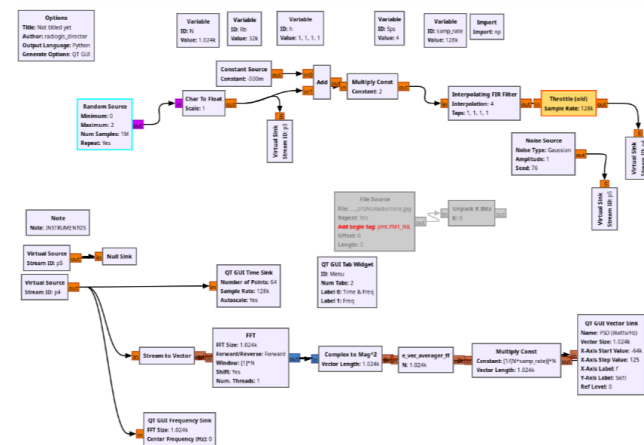


Flujograma para archivos reales

Estos archivos los leímos y procesamos con los bloques correspondientes, lo que nos permitió observar cómo se comportan esas señales tanto en el dominio del tiempo como en su densidad espectral de potencia (PSD). Fue interesante ver la diferencia entre trabajar con la imagen y con el audio, porque cada tipo de archivo mostró comportamientos distintos y nos ayudó a entender mejor cómo se representan y analizan en el sistema.

3 Resultados

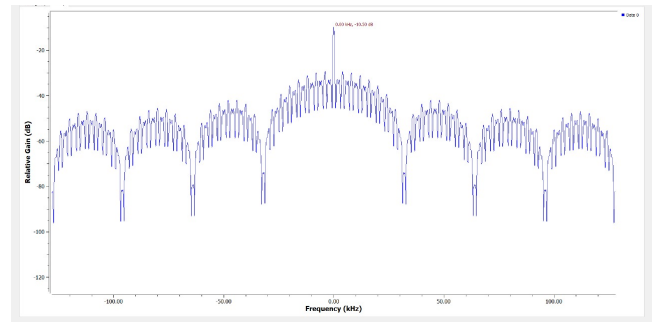
Punto 2. Cuando el SPS (Samples per Symbol) se configura con longitud $H = 1$, este valor corresponde a un tren de impulsos unitario. Al convolucionar con la señal de entrada aleatoria (random), el resultado conserva únicamente la aleatoriedad propia de dicha entrada, generando así una señal que se comporta como ruido tanto en el dominio del tiempo como en el de la frecuencia. Esto ocurre porque, al ser H un vector de longitud uno, no aporta ningún tipo de filtrado



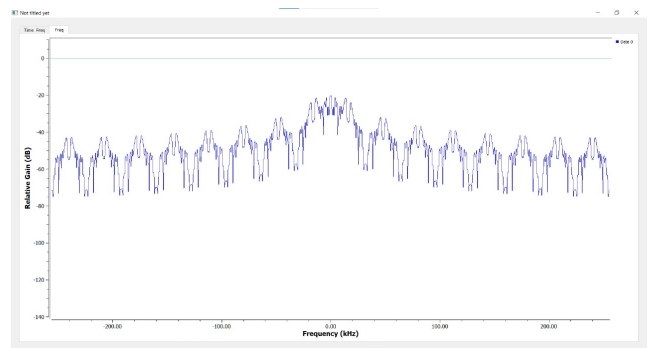
Flujograma para la señal aleatoria.

ni forma de pulso, por lo que la salida refleja directamente la naturaleza aleatoria de la secuencia de entrada.

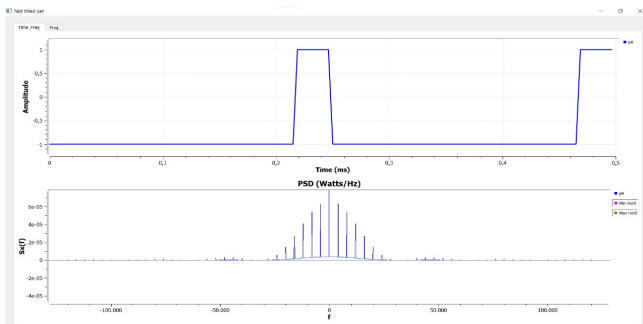
En las simulaciones realizadas en GNU Radio se analizó el efecto del parámetro SPS (Samples per Symbol) sobre la forma de la señal y su densidad espectral de potencia (PSD). Con un SPS de 4, el tren de impulsos es muy corto, lo que genera variaciones abruptas en el tiempo y una PSD con múltiples picos, sin una envolvente suavizada; además, aparece un impulso marcado en la frecuencia cero y lóbulos laterales más amplios. Al aumentar el SPS a 8, el tren de impulsos se alarga, lo que produce bloques rectangulares más definidos en el dominio temporal y una PSD más limpia y estable, en la que desaparece la componente en DC debido a la mayor dispersión de la energía en frecuencia. Finalmente, con un SPS de 16 estos efectos se acentúan: la representación temporal muestra intervalos más largos, la PSD en frecuencia cero incrementa su valor, y la distribución espectral se vuelve más uniforme, con un mayor número de componentes en frecuencia que reflejan la mayor duración del tren de impulsos. En conjunto, se confirma que un mayor SPS suaviza la señal y mejora la claridad de la PSD, reduciendo la apariencia de ruido en el espectro.



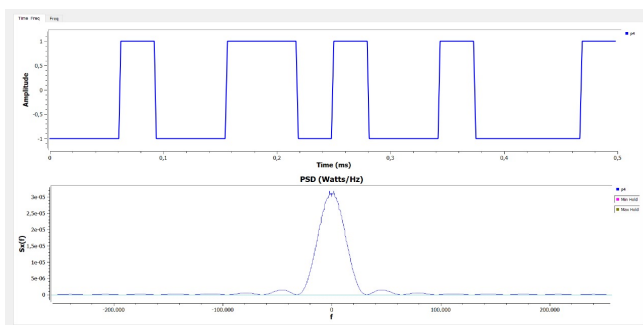
Gráfica en frecuencia con sps=4.



Gráfica en frecuencia con sps=16.

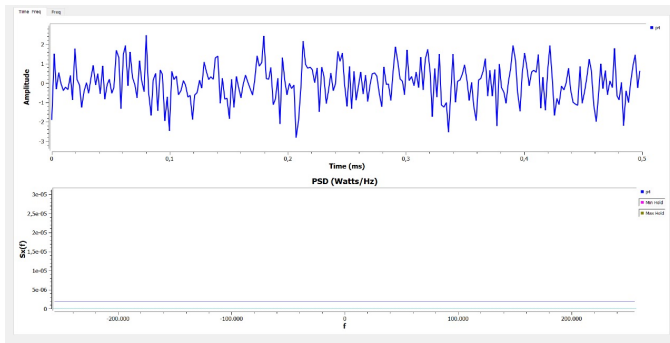


Gráfica en el tiempo con Sps=4.

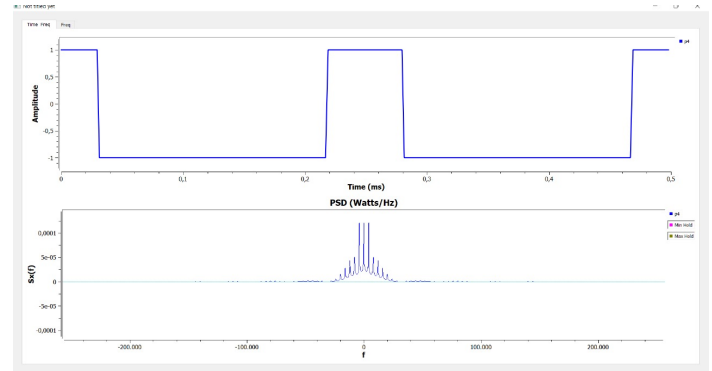


Gráfica en el tiempo con sps=16.

Punto 3. En la prueba realizada, al redirigir las señales a través de las Virtual Source configuradas como p4 y p5, se observó en el dominio temporal un comportamiento aleatorio característico del ruido blanco. En el dominio de la frecuencia, la densidad espectral de potencia (PSD) mostró un nivel constante en Watts/Hz, lo cual confirma que la energía del ruido blanco se distribuye uniformemente en el espectro. Este resultado es consistente con la teoría: al promediar realizaciones sucesivas, las fluctuaciones aleatorias tienden a estabilizarse en un valor medio constante. El nivel observado depende de la ganancia o amplitud configurada en la fuente de ruido, y en caso de que la señal no esté centrada en cero, aparece una componente en DC adicional, pero la característica esencial de PSD plana se mantiene.

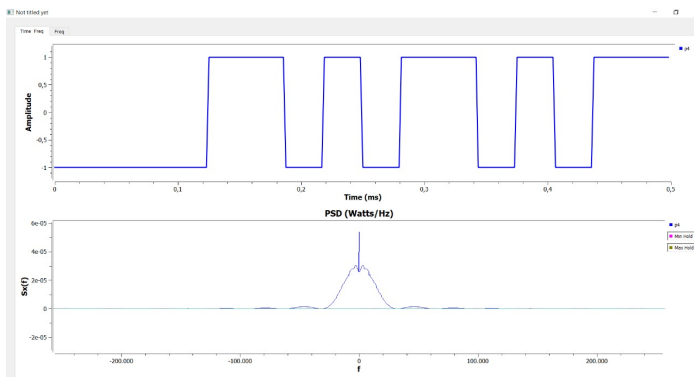


Gráficas del Ruido en el tiempo y psd



Gráfica enviando archivo WAV.

Punto 4 y 5. En la experimentación con archivos reales se evidenció un comportamiento distinto de la densidad espectral de potencia (PSD) según el tipo de señal de entrada. Con el archivo JPG, la codificación binaria generó una fuerte componente de continua en frecuencia cero, reflejada en un pico dominante en la PSD, lo que concentra gran parte de la energía en DC y produce un espectro menos uniforme. En contraste, el archivo WAV (audio) mostró una distribución de energía más balanceada en frecuencia, con múltiples lóbulos y picos que se mantienen incluso al variar el número de símbolos por muestra (SPS), destacando que el incremento de SPS solo afina la forma del sinc pero no concentra la energía únicamente en cero. Esto se explica teóricamente por la diferencia en la estructura estadística de cada señal: las secuencias binarias con corridas largas tienden a reforzar la componente en DC, mientras que el audio presenta variaciones continuas que distribuyen mejor su potencia espectral.



Gráfica enviando archivo JPG.

Respuestas del Cuestionario de Laboratorio

a. Esa combinación (Constant Source = -0.5 ; Add; Multiply Const = 2) la usamos para mapear los bits de $\{0,1\}$ a $\{-1,+1\}$. En la práctica hicimos $(\text{bit} - 0.5) \times 2 = 2 \cdot \text{bit} - 1$. Con eso, la señal quedó balanceada alrededor de cero y lista para la modulación/filtrado.

b. El Interpolation FIR Filter nos sirvió para poner Sps muestras por símbolo y ‘dar forma’ a cada bit. Lo dejamos con Interpolation = Sps y con taps rectangulares ($h = [1,1,\dots,1]$).

1. Interpolation = Sps para que cada símbolo tenga exactamente Sps muestras; si pongo otro valor, se descuadra la relación tiempo/frecuencia y la PSD se corre.

2. Para ver la señal en p3, añadimos un “Virtual Sink/Source” en p3 y lo conectamos a un QT Time/Frequency Sink con el sample rate correspondiente a p3.

3. En p4 el analizador muestra un ancho total igual a la frecuencia de muestreo en p4, así que usamos: $BW_{p4} (\text{mostrado}) = Fs_{p4} = R_b \times Sps$.

4. La frecuencia de muestreo en p3 la calculamos a partir de p4: $Fs_{p3} = Fs_{p4} / Sps$.

c. Al comparar PSD, la de la binaria que salió de audio no se comportó igual que la que salió de la foto porque el patrón de bits (densidad de transiciones, corridas de 1s/0s, etc.) fue distinto. Esa ‘estructura’ cambia la energía por frecuencias aunque ambas sean bipolares rectangulares.

d. Throttle actuó como ‘limitador de velocidad’. Es más que todo para evitar que el PC y CPU procesen a tope. Con Throttle fijamos la tasa y la visualización quedó estable.

e. Cuando dejamos la señal en $\{0,1\}$ (sin pasar a $\{-1,+1\}$), en la PSD apareció un componente DC grande (pico en $f=0$) y más energía en bajas frecuencias. Al hacerla

bipolar, ese DC desapareció y el espectro quedó más simétrico.

f. El ruido blanco ‘ideal’ es infinito en banda, pero en el laboratorio solo vimos una banda plana dentro de $\pm F_s/2$. Es normal: el analizador solo muestra lo que está muestreado; fuera de ese rango no hay información.

g. La binaria rectangular teórica es de banda infinita; en la práctica, otra vez, solo se vio lo que cupo en $\pm F_s/2$ y además el propio filtrado (taps) acotó más la energía.

h. Para contar lóbulos nos sirvió que la separación entre ceros es R_b . En todo el ancho del analizador (F_s) caben aproximadamente F_s/R_b lóbulos. Como $F_s = R_b \times Sps$, el conteo total (dos lados, contando el lóbulo central como dos) nos dio $\approx Sps$.

i. El rango total que mostró el espectro lo sacamos con: $F_s = R_b \times Sps$. Eso define el eje de frecuencias del Frequency Sink ($\pm F_s/2$).

j. La resolución espectral del analizador la calculamos como $\Delta f = F_s / N$, donde N es el tamaño de FFT. Efectivamente, al subir N , vimos más detalle.

k. Al poner $K = 16$ en “Unpack K Bits”, la salida del bloque entregó 16 bits por cada elemento de entrada (en vez de 8). Eso duplicó el caudal de bits respecto a $K=8$; si no se ajustan los ‘sample rates’ de los sinks, el eje de frecuencia queda mal escalado.

l. Para F_s a la entrada del Unpack K Bits, partimos de lo que observamos en la PSD: con L lóbulos visibles y un ancho de señal BW_{sign} (por ejemplo, usando R_b o el primer nulo), sacamos R_b y luego $F_{s_entrada} = R_b$ (del flujo antes de desempaquetar). En términos prácticos, usando que $L \approx Sps$ y $R_b \approx$ (ancho hasta el primer nulo), reconstruimos $F_{s_entrada} = R_b$.

m. La frecuencia de muestreo a la salida del Unpack K Bits la obtuvimos como $F_{s_salida} = K \times F_{s_entrada}$ (porque por cada elemento de entrada se generan K bits a la salida).

n. En “Char To Float” confirmamos que la tasa no cambia: $F_{s_salida} = F_{s_entrada}$. Ese bloque solo convierte el tipo de dato.

o. La PSD de la binaria bipolar se nos pareció a la de un ruido blanco cuando usamos $Sps = 1$. Con $Sps=1$ la secuencia ± 1 queda i.i.d. a la tasa de muestreo y su PSD

discreta se ve prácticamente plana.

p. Para que cada bit tuviera forma de ‘diente de sierra’, cambiamos los taps h por una rampa normalizada dentro de un símbolo: $h[n] = n$ para $n=0 \dots Sps-1$ (escalada a $[0,1]$) y dejamos Interpolation = Sps . Eso hizo que cada símbolo subiera linealmente y reiniciará en el siguiente.

q. Para Unipolar RZ (Fig. 3) quitamos la conversión a bipolar (nos quedamos en $\{0,1\}$) y en el FIR pusimos h con 50% de duty: $h = [1, \dots, 1, 0, \dots, 0]$ (primer $Sps/2$ unos, segundo $Sps/2$ ceros). Interpolation quedó en Sps .

r. Para Manchester (Fig. 3) Mantuvimos la señal bipolar y elegimos h con dos niveles dentro del símbolo: $h = [+1, \dots, +1, -1, \dots, -1]$ (mitad positiva y mitad negativa). Así cada bit tuvo una transición a la mitad del período.

s. Para OOK (Fig. 4) trabajamos en unipolar: dejamos la señal en $\{0,1\}$ y taps rectangulares $h = [1, \dots, 1]$. Si queríamos verla en pasabanda como en la figura, multiplicamos luego por una portadora $\cos(2\pi f_{ct})$ con un bloque Multiply.

t. Para BPSK (Fig. 4) usamos la bipolar $\{-1, +1\}$ y después la multiplicamos por una cosenoidal a f_{ct} : $s(t) = A \cdot m(t) \cdot \cos(2\pi f_{ct} t)$. Con h rectangular ya se vio como BPSK en el osciloscopio/analizador.

u. Para ASK (Fig. 5) nos quedamos en unipolar y ajustamos amplitudes: ‘0’ $\rightarrow 0$ y ‘1’ $\rightarrow A$ (o dos niveles $A1/A2$). Con h rectangular y (opcionalmente) multiplicando por $\cos(2\pi f_{ct})$ se apreció la modulación ASK.

v. Para que pareciera ‘latido’ (según la figura), diseñamos h con un pico angosto y pos-ondulación (tipo ECG simplificado): un tren corto positivo, seguido de una pequeña parte negativa y retorno a cero, todo dentro de Sps . Normalizamos h y mantuvimos Interpolation = Sps .

w. Para los ‘pulsos rizados’ (Fig. 7) construimos h como una senoidal amortiguada dentro del símbolo: $h[n] \approx e^{-\alpha n} \cdot \sin(2\pi f_r n)$, $0 \leq n < Sps$, y la normalizamos. Eso generó la señal característica.

x. En PSD, la unipolar mostró un pico fuerte en DC y más energía en bajas frecuencias; la bipolar no tiene componente en DC (cero en $f=0$) y reparte mejor la energía. la unipolar ‘ensucia’ la zona baja; la bipolar es más ‘limpia’ en el centro.



4 Conclusiones

Con esta práctica entendimos que no basta con generar señales, sino que los parámetros que uno ajusta cambian por completo los resultados. Al mover el *Samples per Symbol* vimos cómo la señal podía pasar de verse desordenada a mostrarse mucho más clara, y eso nos dejó la idea de que un detalle pequeño puede marcar una gran diferencia.

El ruido blanco fue de lo más revelador. En el tiempo parecía algo sin forma, como un desorden total, pero al verlo en frecuencia se confirmó que en realidad era uniforme. Esa simple observación nos ayudó a aterrizar la teoría y a ver que lo que parecía “ruido sin sentido” tiene una lógica muy clara cuando se analiza de otra manera.

Al probar con la imagen y con el audio descubrimos que cada señal tiene su propia huella. La foto se concentró en un solo punto del espectro, mientras que el audio mostró muchas variaciones. Eso nos dejó pensando en que la naturaleza de los datos cambia totalmente cómo se comportan en frecuencia, y que no hay un único patrón que sirva para todo.

Algo que también valoramos fue darnos cuenta de que los bloques del flujograma cada uno cumple un papel clave. Sin el *Throttle* la simulación se habría vuelto inestable, y sin el *Interpolation FIR Filter* no habríamos entendido bien la forma de la señal.

Más allá de las gráficas y los cálculos, lo más importante fue que la práctica nos permitió experimentar, equivocarnos y volver a probar. Sentimos que de verdad conectamos la teoría con la práctica y que logramos apropiarnos de los conceptos. Al final nos quedamos con la satisfacción de que lo que vimos en el laboratorio no fue solo un requisito académico, sino un aprendizaje útil para nuestra formación.

5 Referencias

- [1] H. O. Boada and O. M. R. Torres, *Comunicaciones Plus: Volumen II*, Bucaramanga, Colombia: Editorial UIS, 2022.



Universidad Industrial de Santander
Escuela de Ingenierías Eléctrica,
Electrónica y de
Telecomunicaciones

Laboratorio de comunicaciones II (27139)
Practica 1: PROGRAMACION EN RADIO
DEFINIDA POR SOFTWARE (GNURADIO)
GRUPO: A1 - G2





Universidad Industrial de Santander
Escuela de Ingenierías Eléctrica,
Electrónica y de
Telecomunicaciones

Laboratorio de comunicaciones II (27139)
Practica 1: PROGRAMACION EN RADIO
DEFINIDA POR SOFTWARE (GNURADIO)
GRUPO: A1 - G2





Universidad Industrial de Santander
Escuela de Ingenierías Eléctrica,
Electrónica y de
Telecomunicaciones

Laboratorio de comunicaciones II (27139)
Practica 1: PROGRAMACION EN RADIO
DEFINIDA POR SOFTWARE (GNURADIO)
GRUPO: A1 - G2

